

Section 1 — Spread Operator (10 Problems)

Q1. Copy elements of one array into another using the spread operator.

Sample Input:

```
const arr1 = [10, 20, 30];
```

Sample Output:

```
New Array: [10, 20, 30]
```

Q2. Merge two arrays into one using the spread operator.

Sample Input:

```
const a = [1, 2];  
const b = [3, 4];
```

Sample Output:

```
Merged Array: [1, 2, 3, 4]
```

Q3. Insert elements at the beginning and end of an array using spread.

Sample Input:

```
const arr = [2, 3, 4];
```

Sample Output:

```
Updated Array: [1, 2, 3, 4, 5]
```

Q4. Call a function by spreading an array of arguments.

Sample Input:

```
function add(a, b, c) {  
  return a + b + c;  
}  
const nums = [10, 20, 30];
```

Sample Output:

```
Sum = 60
```

Q5. Clone an object using spread syntax.

Sample Input:

```
const student = { name: "Ravi", age: 21 };
```

Sample Output:

```
Cloned Object: { name: 'Ravi', age: 21 }
```

Q6. Merge two objects using spread operator.

Sample Input:

```
const obj1 = { id: 1, name: "Amit" };  
const obj2 = { city: "Pune", marks: 90 };
```

Sample Output:

```
Merged Object: { id: 1, name: 'Amit', city: 'Pune', marks: 90 }
```

Q7. Demonstrate overwriting of properties using spread syntax.

Sample Input:

```
const base = { id: 1, name: "Raj" };  
const update = { name: "Rahul" };
```

Sample Output:

```
Result: { id: 1, name: 'Rahul' }
```

Q8. Convert a string into an array of characters using spread operator.

Sample Input:

```
const str = "CJC";
```

Sample Output:

```
['C', 'J', 'C']
```

Q9. Use spread with Math functions to find max number.

Sample Input:

```
const numbers = [11, 56, 72, 4, 33];
```

Sample Output:

```
Maximum Number: 72
```

Q10. Prove that spread creates a shallow copy of an object.

Sample Input:

```
const user = { name: "Ravi", address: { city: "Pune" } };  
const copy = { ...user };  
copy.address.city = "Mumbai";
```

Sample Output:

```
Original: { name: 'Ravi', address: { city: 'Mumbai' } }
```

Section 2 — Rest Parameter (10 Problems)

Q1. Accept any number of arguments using rest parameter and display them.

Sample Input:

```
display(10, 20, 30, 40);
```

Sample Output:

```
Arguments: [10, 20, 30, 40]
```

Q2. Create a function that sums all numbers passed using rest parameter.

Sample Input:

```
sumAll(10, 20, 30);
```

Sample Output:

```
Total Sum: 60
```

Q3. Print a person's name and multiple skills using rest parameter.

Sample Input:

```
introduce("Ravi", "HTML", "CSS", "JS");
```

Sample Output:

```
Name: Ravi  
Skills: HTML, CSS, JS
```

Q4. Write a function that counts the total number of arguments.

Sample Input:

```
countArgs("A", "B", "C");
```

Sample Output:

```
Total Arguments: 3
```

Q5. Join multiple strings using rest parameter.

Sample Input:

```
combine("JavaScript", "is", "awesome");
```

Sample Output:

```
JavaScript is awesome
```

Q6. Use array destructuring and rest to separate the first element and remaining.

Sample Input:

```
const arr = [10, 20, 30, 40];
```

Sample Output:

```
First: 10  
Rest: [20, 30, 40]
```

Q7. Create an arrow function to calculate product of all numbers using rest.

Sample Input:

```
multiply(2, 3, 4);
```

Sample Output:

```
Product: 24
```

Q8. Write a function with one default value and rest parameter to calculate sum.

Sample Input:

```
add(5, 10, 20);
```

Sample Output:

```
Total: 35
```

Q9. Demonstrate difference between spread and rest.

Sample Input:

```
function show(a, b, ...rest) { console.log(a, b, rest); }  
show(...[1, 2, 3, 4, 5]);
```

Sample Output:

```
1 2 [3, 4, 5]
```

Q10. Filter odd numbers using rest parameter.

Sample Input:

```
filterOdds(1, 2, 3, 4, 5, 6);
```

Sample Output:

```
Odd Numbers: [1, 3, 5]
```

Section 3 — Destructuring (10 Problems)

Q1. Perform array destructuring to extract values.

Sample Input:

```
const arr = [10, 20, 30];
```

Sample Output:

```
a = 10, b = 20, c = 30
```

Q2. Skip certain array elements while destructuring.

Sample Input:

```
const arr = [1, 2, 3];
```

Sample Output:

```
a = 1, b = 3
```

Q3. Use default values during array destructuring.

Sample Input:

```
const [name = "Guest", age = 20] = [];
```

Sample Output:

```
name = Guest, age = 20
```

Q4. Swap variable values using array destructuring.

Sample Input:

```
let a = 5, b = 10;
```

Sample Output:

```
Before Swap: a=5, b=10  
After Swap: a=10, b=5
```

Q5. Perform nested array destructuring.

Sample Input:

```
const arr = [1, [2, 3], 4];
```

Sample Output:

```
a=1, b=2, c=3, d=4
```

Q6. Extract properties from an object using destructuring.

Sample Input:

```
const student = { name: "Amit", age: 22 };
```

Sample Output:

```
name = Amit, age = 22
```

Q7. Rename properties during object destructuring.

Sample Input:

```
const person = { fname: "Ravi", city: "Pune" };
```

Sample Output:

```
fullName = Ravi, city = Pune
```

Q8. Perform nested object destructuring.

Sample Input:

```
const emp = { id: 1, details: { dept: "IT", location: "Mumbai" } };
```

Sample Output:

```
dept = IT, location = Mumbai
```

Q9. Use object destructuring inside function parameters.

Sample Input:

```
function show({ name, age }) {  
  console.log(`${name} is ${age} years old.`);  
}
```

```
show({ name: "Kunal", age: 25 });
```

Sample Output:

Kunal is 25 years old.

Q10. Use rest syntax in object destructuring.**Sample Input:**

```
const obj = { a: 1, b: 2, c: 3 };  
const { a, ...rest } = obj;
```

Sample Output:

```
a = 1  
rest = { b: 2, c: 3 }
```
